

Fundamentos de Inteligência Artificial

Trabalho Prático N^o 2: The evolution of Lunar Lander

Guilherme Moreira – 2023222537

João Franciso – 2023228417

Turma Prática: PL7

Maio de 2026

Resumo

Este relatório detalha o desenvolvimento e a análise de um sistema de neuroevolução para o simulador Lunar Lander. O objetivo central é a otimização de uma rede neuronal capaz de aterrar uma nave espacial em segurança, tanto em ambientes estáticos como sob a influência de ventos laterais. Através da implementação de um Algoritmo Evolucionário, explorámos diferentes configurações de operadores genéticos e, crucialmente, desenhámos funções objetivo que permitem ao agente aprender manobras complexas de estabilização. Os resultados demonstram que a precisão da aterragem é diretamente proporcional à qualidade da modelação da função de fitness.

Conteúdo

1	Introdução	3
2	Modelação do Algoritmo Evolucionário	3
2.1	Escolha de Operadores	3
3	Implementação das Funções Objetivo	4
3.1	Função Objetivo - Meta 1 (Ambiente Estático)	4
3.2	Função Objetivo - Meta 2 (Ambiente com Vento)	5
4	Análise de Resultados Meta 1	7
4.1	Estudo da População	7
4.2	Experiências da Tabela 2	7
5	Análise de Resultados Meta 2	9
5.1	A Fragilidade da Baseline	9
5.2	Resultados com a Nova Função Objetivo	9
6	Conclusão	11

1 Introdução

O Lunar Lander é um problema clássico de controlo onde um agente deve gerir motores para pousar suavemente numa zona específica. Neste projeto, a abordagem utilizada é a neuroevolução, na qual um Algoritmo Evolucionário otimiza os pesos de uma rede neuronal. O controlador recebe um vetor de 8 observações (incluindo posição, velocidade e ângulo) e produz 2 ações (aceleração vertical e lateral).

Dividimos este relatório na análise dos operadores genéticos, no detalhe da implementação das funções objetivo e numa discussão profunda dos resultados experimentais obtidos em ambas as metas do projeto.

2 Modelação do Algoritmo Evolucionário

2.1 Escolha de Operadores

A eficácia da evolução depende da capacidade de explorar o espaço de pesos da rede sem destruir as soluções promissoras. Com base em testes preliminares (Figura 1), definimos os seguintes operadores:

- **Seleção (Torneio):** Seleccionamos o melhor entre 3 indivíduos aleatórios. Esta técnica mantém uma pressão seletiva equilibrada.
- **Crossover (Uniforme):** Permite que o descendente herde pesos de camadas diferentes de cada progenitor, o que é ideal para redes neuronais onde a ordem dos genes não tem uma relação espacial rígida.
- **Mutação (Gaussiana):** Com um desvio padrão de 0.1, permite pequenos ajustes nos pesos, fundamentais para a afinação fina necessária na aterragem.

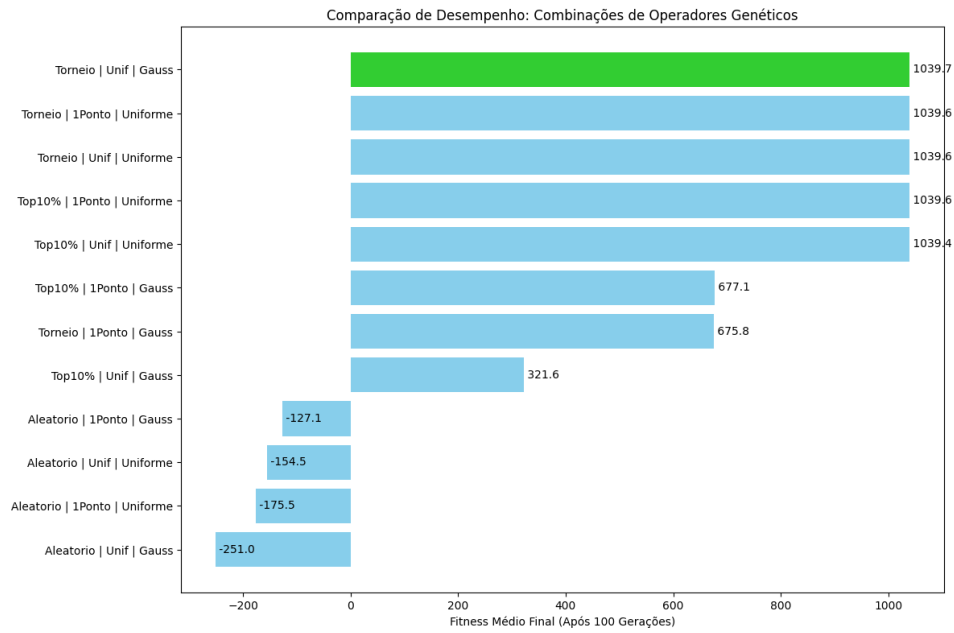


Figura 1: Gráfico de comparação de desempenho entre diferentes combinações de operadores genéticos.

3 Implementação das Funções Objetivo

A função objetivo é o componente mais crítico do sistema. Ela deve fornecer um sinal de aprendizagem que recompense o progresso, mesmo quando o agente ainda não consegue aterrar com sucesso.

3.1 Função Objetivo - Meta 1 (Ambiente Estático)

Para o ambiente sem vento, focámo-nos na distância final, na velocidade de impacto e na postura da nave. O código implementado foi:

Listing 1: Função Objetivo utilizada na Meta 1

```

1 def objective_function(observation_history):
2     final_obs = observation_history[-2]
3     x, y, vx, vy, angle, v_angle, left_leg, right_leg =
4         final_obs
5
6     # Distancia Euclidiana para aproximacao suave
7     distance = np.sqrt(x**2 + y**2)

```

```

7     fitness = -distance * 100
8
9     # Penalizacao de velocidade para aprender a travar
10    speed = np.sqrt(vx**2 + vy**2)
11    fitness -= speed * 100
12
13    # Penalizacao de inclinacao e velocidade angular
14    fitness -= abs(angle) * 100
15    fitness -= abs(v_angle) * 50
16
17    # Recompensa por contacto das pernas
18    fitness += (left_leg + right_leg) * 20
19
20    success = check_successful_landing(final_obs)
21    if success:
22        fitness += 1000
23    return fitness, success

```

Análise Técnica: Esta função utiliza a distância euclidiana para criar um gradiente de fitness contínuo. Ao penalizar a *speed*, forçamos o algoritmo a favorecer redes que ativam o motor principal durante a descida. O bónus de 1000 pontos serve como um catalisador para que as soluções próximas do sucesso dominem rapidamente a população.

3.2 Função Objetivo - Meta 2 (Ambiente com Vento)

Com o vento ligado, a nave é empurrada constantemente para os lados. A função da Meta 1 revelou-se insuficiente para ensinar a nave a contrariar esta força. Evoluímos a função para recompensar o esforço contínuo de combate ao vento:

Listing 2: Função Objetivo com bónus de combate ao vento

```

1 def objective_function(observation_history):
2     # Vamos buscar o estado final (antes do choque/pouso)
3     final_obs = observation_history[-2]
4     x, y, vx, vy, angle, v_angle, left_leg, right_leg =
        final_obs
5
6     # Avaliacao Base (Focada no Pouso)

```

```

7     distance = np.sqrt(x**2 + y**2)
8     fitness = -distance * 100
9
10    speed = np.sqrt(vx**2 + vy**2)
11    fitness -= speed * 100
12
13    fitness -= abs(angle) * 100
14    fitness -= abs(v_angle) * 50
15    fitness += (left_leg + right_leg) * 20
16
17    # Avaliacao de Trajetoria
18    # Premiamos a permanencia no centro.
19    estabilidade_trajetoria = 0
20    for obs in observation_history:
21        # Recompensa por frame se estiver no corredor
22        # central (X entre -0.3 e 0.3)
23        if abs(obs[0]) < 0.3:
24            estabilidade_trajetoria += 2.0
25        else:
26            # Penaliza se for arrastado pelo vento para fora
27            # do corredor
28            estabilidade_trajetoria -= 1.0
29
30    fitness += estabilidade_trajetoria
31
32    # Bonus de Sucesso
33    success = check_successful_landing(final_obs)
34    if success:
35        fitness += 1000
36
37    return fitness, success

```

Análise Técnica: O vento exerce uma aceleração constante. Para o anular, a nave deve manter uma inclinação negativa e usar o motor principal para gerar um vetor de força lateral à esquerda. Ao iterar sobre o `observation_history`, recompensamos o agente por manter esta postura defensiva durante todo o voo, e não apenas no final.

4 Análise de Resultados Meta 1

4.1 Estudo da População

A Figura 2 mostra que populações muito pequenas (10-30) não possuem diversidade genética suficiente para encontrar a plataforma. O "cotovelo" da curva ocorre nos 100 indivíduos, onde o fitness estabiliza.

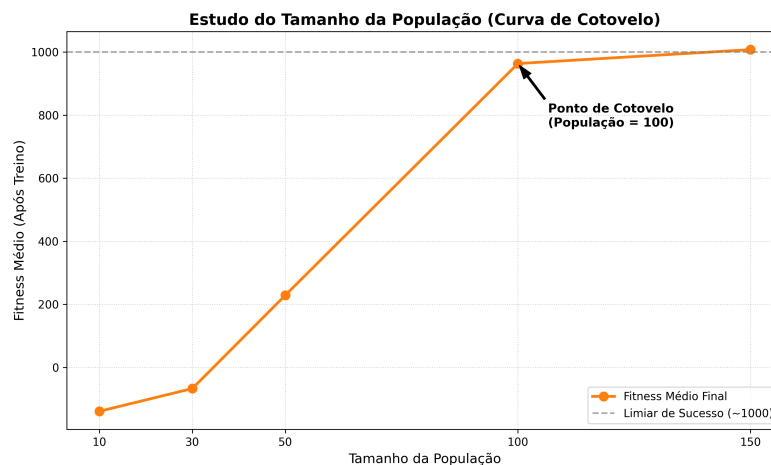


Figura 2: Análise de escalabilidade do algoritmo face ao tamanho da população.

4.2 Experiências da Tabela 2

As 8 experiências obrigatórias demonstraram que o crossover de 0.9 é o motor principal do sucesso. Sem elitismo, o algoritmo explora melhor o espaço, mas com elitismo (Exp 5-8), o desvio padrão aumenta, indicando que a população pode ficar presa em soluções medíocres mas estáveis. A Experiência 3 obteve a melhor performance com **97% de sucesso**.

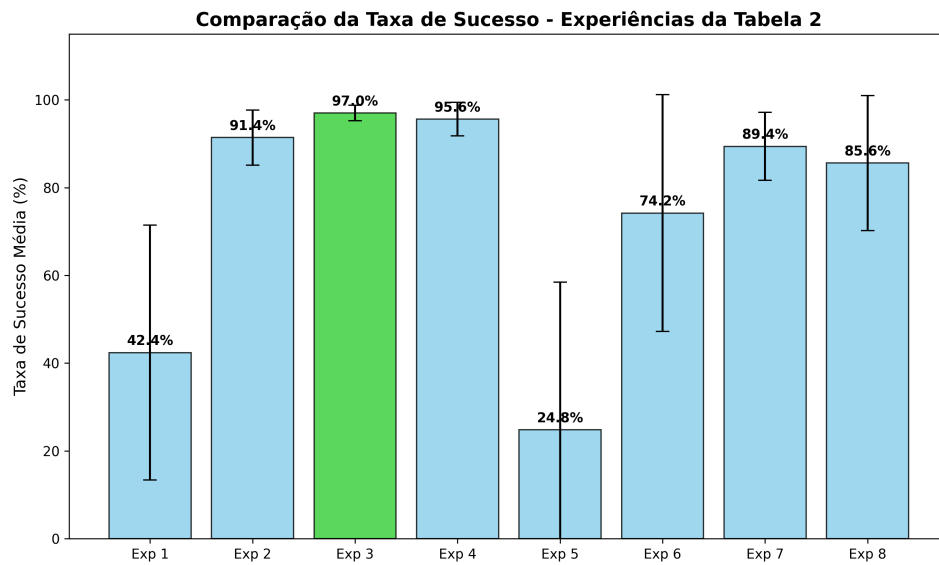


Figura 3: Comparação das taxas de sucesso médias com desvio padrão (Meta 1).

O comportamento evolutivo deste agente ótimo pode ser visualizado através da sua curva de aprendizagem (Figura 4), onde notamos uma subida acentuada nas primeiras gerações, seguida de uma forte estabilização.

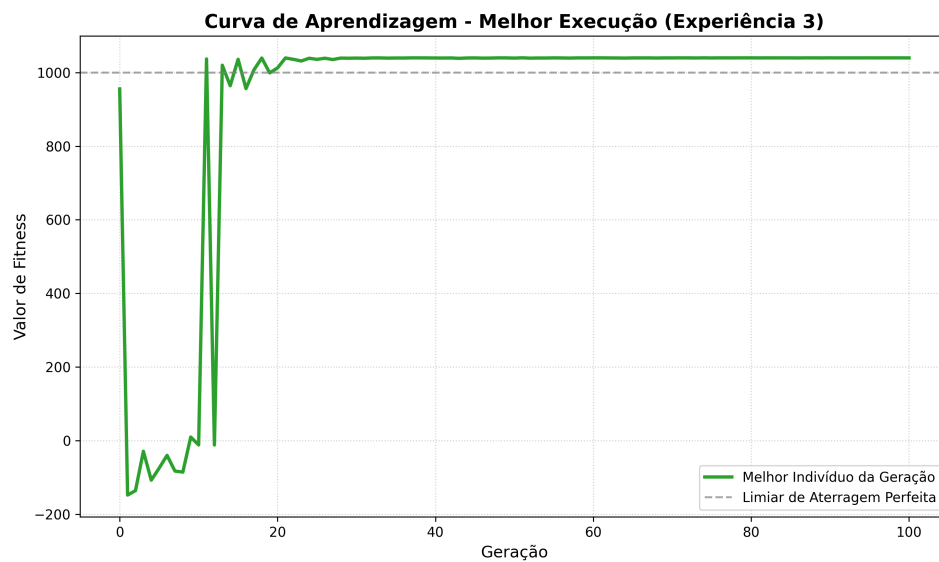


Figura 4: Curva de aprendizagem do melhor indivíduo em ambiente estático (Meta 1).

5 Análise de Resultados Meta 2

5.1 A Fragilidade da Baseline

Quando aplicámos o melhor indivíduo da Meta 1 ao ambiente com vento, a performance caiu drasticamente para **58.2%** de taxa de sucesso (Figura 5). Isto prova que um controlador otimizado para ambientes estáticos não possui a flexibilidade necessária para ambientes estocásticos sem um sinal de treino específico para as novas perturbações.

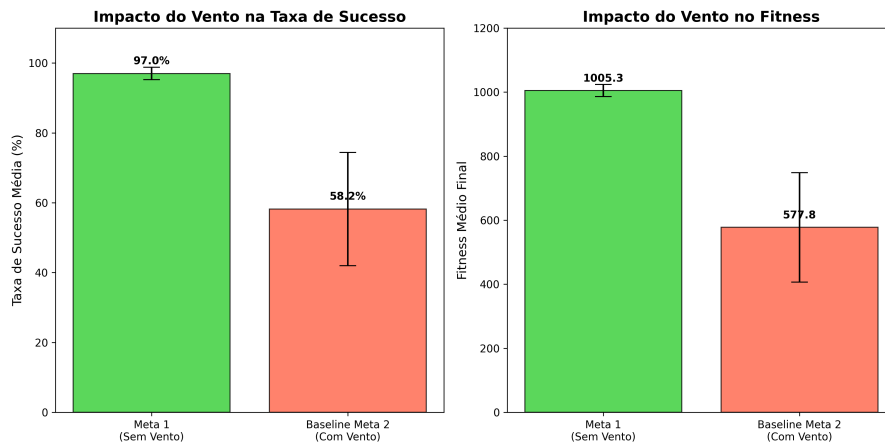


Figura 5: Comparação entre o desempenho original e a baseline perante o vento.

5.2 Resultados com a Nova Função Objetivo

Após introduzirmos o bônus de combate ao vento e aumentarmos o treino para 150 gerações, a performance estabilizou. A Figura 6 mostra que conseguimos atingir uma **média de 72.8% de sucesso**, com picos de 80% em várias runs.

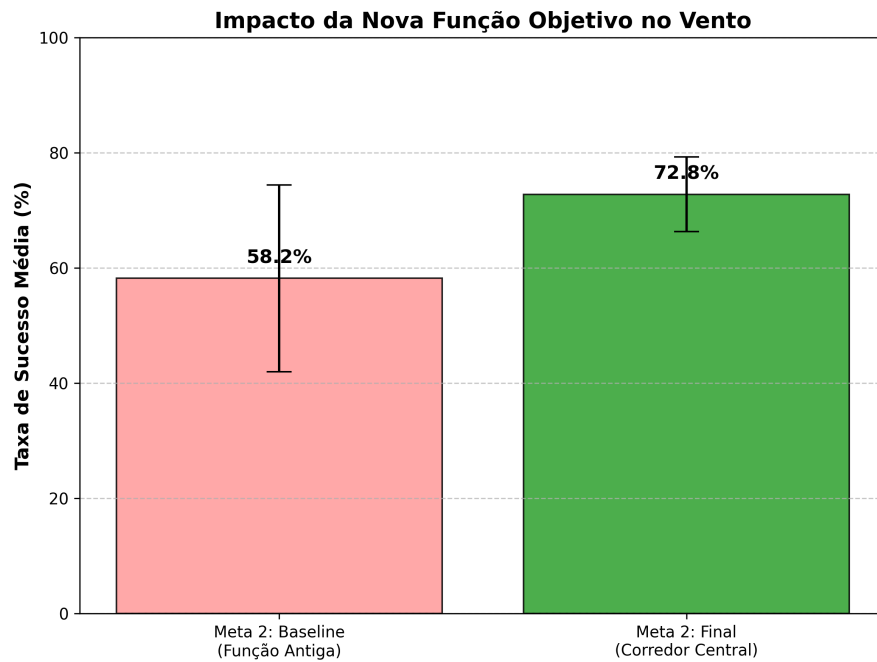


Figura 6: Comparação da Taxa de Sucesso Média na Meta 2 (Baseline vs Função Final).

Nesta figura 7 conseguimos verificar que a nova objective function permitiu um aumento do desempenho e estabilização do desvio padrão, graças ao combate ao vento.

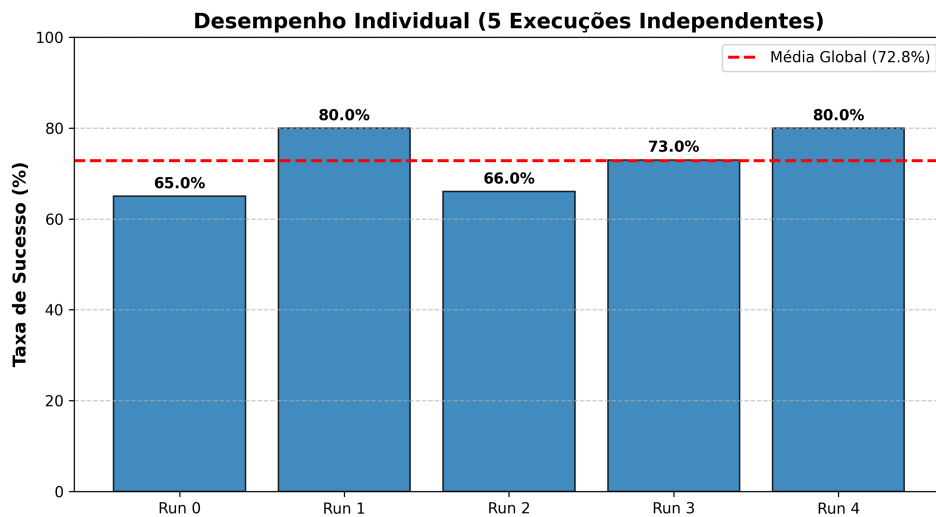


Figura 7: Desempenho individual das 5 execuções independentes na Meta 2.

A Figura 8 ilustra a curva de aprendizagem. Nota-se que o algoritmo demora cerca de 30 gerações para sair dos valores intermédios, comparado com a Meta 1.

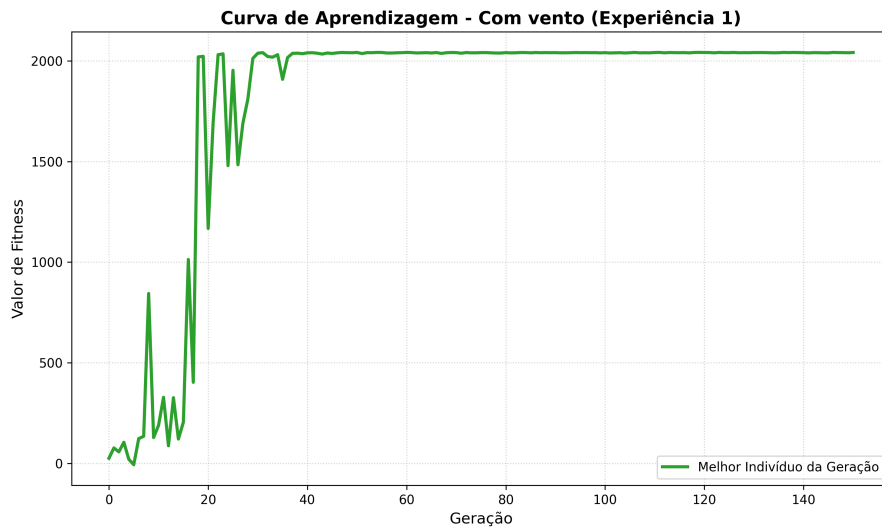


Figura 8: Curva de aprendizagem demonstrando a adaptação ao vento.

6 Conclusão

Este trabalho demonstrou que a Neuroevolução é uma ferramenta poderosa para a automação de controladores complexos. A Meta 1 provou que a parametrização correta dos operadores genéticos é essencial para a precisão. A Meta 2 evidenciou que, em ambientes dinâmicos, a função objetivo deve premiar o comportamento contínuo e a resistência a forças externas. Conseguimos evoluir um piloto capaz de pousar na Lua mesmo sob condições meteorológicas adversas, validando a robustez do algoritmo implementado.